

PROJECT REPORT

Team :

- Anushka Agrawal (2021101023)
- Vanshika Dhingra (2021101092)
- Khushi Wadhwa (2021101104)

Why do we do Pruning?

- Neural networks are often **over-parameterized**, meaning that they have more parameters than they actually need to learn the desired task. This means that many of the weights in the network are not actually doing anything useful. Pruning can help to remove these redundant weights, making the network more efficient.
- Neural networks are often trained using stochastic gradient descent (SGD). SGD can sometimes get stuck in local minima, which can lead to **suboptimal** performance. Pruning can help to prevent SGD from getting stuck in **local minima** by encouraging the network to learn more robust features.
- Pruning can help to improve the **generalization** performance of neural networks. Generalization performance is the ability of a network to perform well on unseen data. Pruning can help to improve generalization performance by removing weights that are not important for the specific task that the network is being trained on.

What does Pruning Do?

During pruning, connections associated with less important or redundant features are removed, leading to a sparser neural network. This can remove the connections based on various saliency criterion or some other methods. So pruning kind of works like feature selection i.e. selecting features most important for the task.

How does retraining help?

During pruning since we remove the connections our accuracy will decrease. Now by retraining the network is encouraged to rely on the remaining important features and adjust their weights to capture the relevant patterns in the data.

What is the difference between structured and unstructured pruning?

- Structured pruning involves pruning entire neuron, filters etc.
- Unstructured Pruning involves pruning individual parameters based on some saliency criteria.

What we have done through our whole study is unstructured pruning

Our Task

The task which we are working on is **Neural Machine Translation**. It involves translating a sentence X of one language(source language) to sentence y of another language(target language).

Encoder Decoder Architecture:

An encoder computes representations for each source sentence. Decoder generates one target word at a time.

Alignment: Word level correspondence between source sentence x and target sentence y .

What we calculate in **NMT** is:

$$p(y | x) = p(y_1 | s) \cdot p(y_2 | y_1, s) \cdot \dots \cdot p(y_T | y_{T-1}, y_{T-2}, \dots, y_1, s)$$

Beam Search Decoding: On each decoding step we keep track of k most probable translations (which we call hypothesis).

Word Embeddings: Word in sentence are converted into vectors such that these vectors are learned to capture context and similar words used in similar context have more close word embeddings. Basically we get the features of the words in the form of numbers.

LSTM with Attention

The **Long Short-Term Memory (LSTM)** model is a type of Recurrent Neural Network (RNN) designed to overcome the vanishing gradient problem and capture long-term dependencies in sequential data.

Architecture Overview

- **Encoder:** A stack of LSTM layers processes the input sentence word-by-word, producing hidden states at each time step.
- **Attention Mechanism:** Introduced to overcome LSTM's bottleneck of compressing the entire input into a single vector. At each decoder step, the model computes a **context vector** as a weighted sum of encoder hidden states, using attention scores.
- **Decoder:** Another LSTM stack generates the output sentence, conditioning on both the context vector and previous outputs.

Key Properties

- **Sequential Processing:** Tokens are processed one at a time in both encoder and decoder.
- **Temporal Dependency Modeling:** Excels at learning time-step dependencies, useful for language tasks.
- **Memory Cell:** Helps retain long-term information without being washed out.

Transformer Model

The **Transformer** architecture revolutionized NLP by replacing recurrence with **self-attention** mechanisms, enabling models to learn dependencies regardless of distance between tokens.

Architecture Overview

- **Encoder:**

- Multi-head Self-Attention layers allow the model to look at all input positions simultaneously.
- Feedforward layers apply transformations to the attention outputs.
- **Decoder:**
 - Similar architecture with additional **cross-attention** to incorporate encoder outputs.
 - Masked self-attention allows autoregressive generation.
- **Positional Encodings:** Since there's no recurrence, sinusoidal positional encodings are added to input embeddings to preserve order.

Key Properties

- **Parallelization:** Allows faster training due to non-sequential computation.
- **Multi-head Attention:** Learns diverse types of relationships between tokens.
- **Layer Normalization:** Helps with stable and deep training.

BERT

Apart from this , we also used a transformer model with Bert embeddings , To get an idea of whether they had a major impact on the translation task.

Evaluation Measures

1. Perplexity:

Intrinsic measurement of how good a model is. A good model is one which gives high probability to the correct sentence

Perplexity is exponential of average negative log likelihood. Decreasing perplexity is equivalent to increasing probability. Suppose we have 10 items with equal probability then perplexity is 10 showing that the model cannot choose among those 10 and is confused.

$$Perplexity = \frac{1}{\sqrt[N]{P(w_1, w_2, \dots, w_N)}}$$

$$Perplexity = \frac{1}{\sqrt[N]{\prod_{i=1}^N p(w_i)}}$$

After taking log and simplifying we get:

$$Perplexity = e^{-\sum_{i=1}^N \log_e(P(w_i))/N}$$

2. Bleu Score

Given a machine translation it tells how good is the machine translation. It indicates how similar the candidate text is to the reference texts that are provided in the dev or test set.

BLEU Score	Interpretation
< 10	Almost useless
10 - 19	Hard to get the gist
20 - 29	The gist is clear, but has significant grammatical errors
30 - 40	Understandable to good translations
40 - 50	High quality translations
50 - 60	Very high quality, adequate, and fluent translations
> 60	Quality often better than human

$$\text{BLEU} = \underbrace{\min\left(1, \exp\left(1 - \frac{\text{reference-length}}{\text{output-length}}\right)\right)}_{\text{brevity penalty}} \underbrace{\left(\prod_{i=1}^4 \text{precision}_i\right)^{1/4}}_{\text{n-gram overlap}}$$

with

$$\text{precision}_i = \frac{\sum_{\text{snt} \in \text{Cand-Corpus}} \sum_{i \in \text{snt}} \min(m_{\text{cand}}^i, m_{\text{ref}}^i)}{w_t^i = \sum_{\text{snt}' \in \text{Cand-Corpus}} \sum_{i' \in \text{snt}', m_{\text{cand}}^{i'}}$$

where

- m_{cand}^i is the count of i-gram in candidate matching the reference translation
- m_{ref}^i is the count of i-gram in the reference translation
- w_t^i is the total number of i-grams in candidate translation

- **Brevity Penalty:**

It penalises very short candidates as almost all the words in candidate can be present in reference if the candidate is very short compared to reference.

- **Disadvantages of Bleu Score:**

- Not good at capturing meaning and grammar of a sentence. does not consider long range dependencies.
- No difference between content and function words
- The BLEU metric performs badly when used to evaluate individual sentences as it is a corpus based metric.
- It does not take into consideration order of the n-grams in the reference and candidate sentence

Why does Pruning weights below a threshold work?

During pruning we prune weights that are redundant for our network. Weights with less magnitude are redundant because they have a smaller impact on the output of the network. This is because neural networks use a non-linear activation function, such as the sigmoid or ReLU function, after each layer of weights. These activation

functions squash the input values into a smaller range, which has the effect of attenuating the impact of small weights.

Three types of magnitude pruning schemes we implemented:

- **Class Blind:**

- Sort the parameters of whole model. Find the threshold for pruning such that x% of weights is pruned.

[attachment:de0042c1-58e2-4daa-843d-e159dc6b7184:ExampleScene.mp4](#)

- Sort the parameters of each class and within each class find the threshold for pruning such that x% of weights in all classes of weights are pruned.

[attachment:44a5c9d9-26c1-4f31-a0fb-9b79c3388608:class-uniform.mp4](#)

- **Class Distribution:**

- For each class c , weights with magnitude less than $\lambda\sigma_c$ are pruned. Here, σ_c is the standard deviation of that class and λ is a universal parameter chosen such that in total, x% of all parameters are pruned.

Summary of Strategies

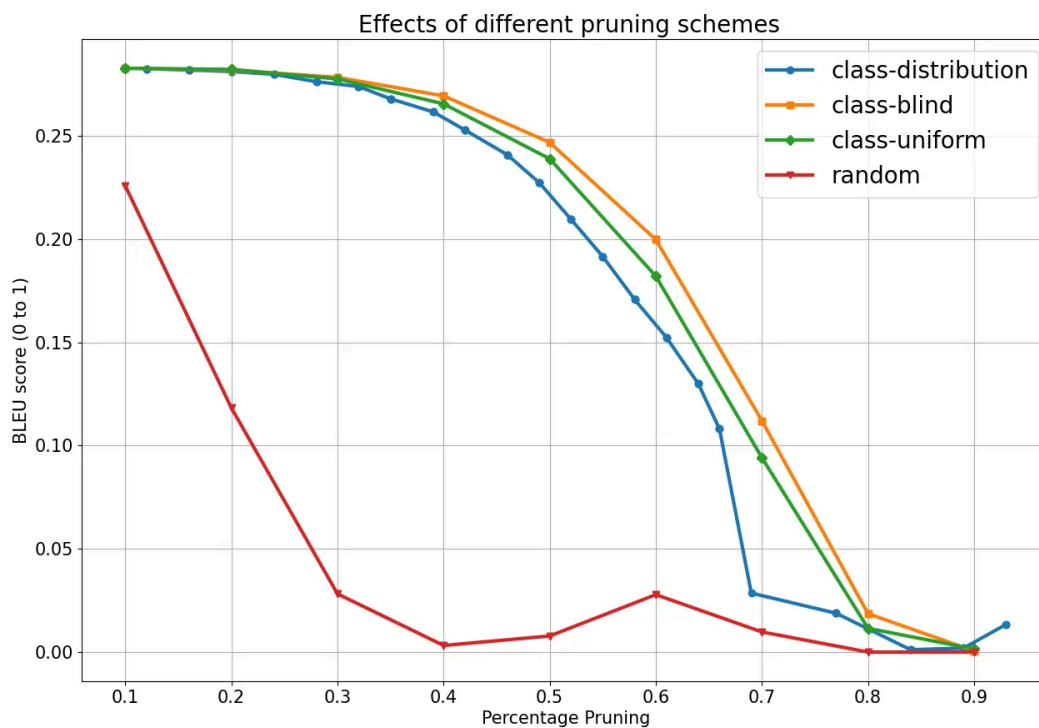
Strategy	Layer Awareness	Balanced Across Layers	Adaptiveness	Tuning Required
Class-Blind	✗	✗	✗	Low
Class-Uniform	✓ (equal treatment)	✓	✗	Medium

Class-Distribution	✅ (statistically)	Adaptive	✅	High
--------------------	-------------------	----------	---	------

RESULTS

LSTM Results

For lstms , We can see that class-blind performs better than both class-uniform and class-distribution.

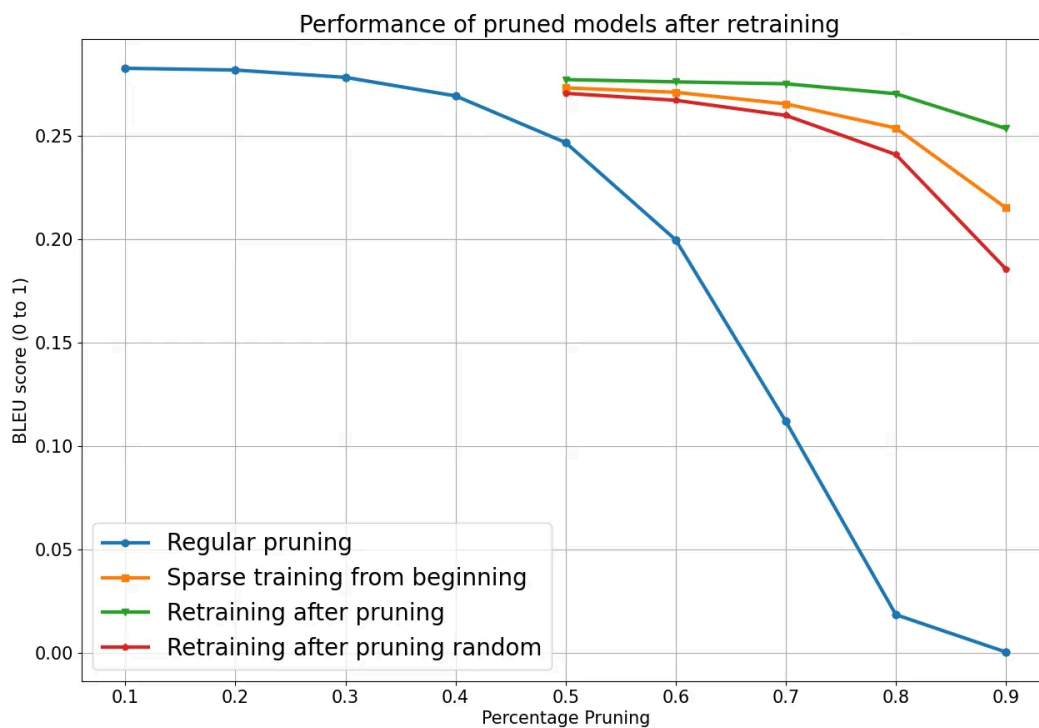


- **Random Global Pruning** - x% of weights are randomly pruned.

Magnitude pruning works better than random pruning, making it clear that weights with less magnitude are more likely to be redundant. All 3 kinds of magnitude pruning outperform random pruning.

Comparison Of Retraining And Sparse From Begin Models:

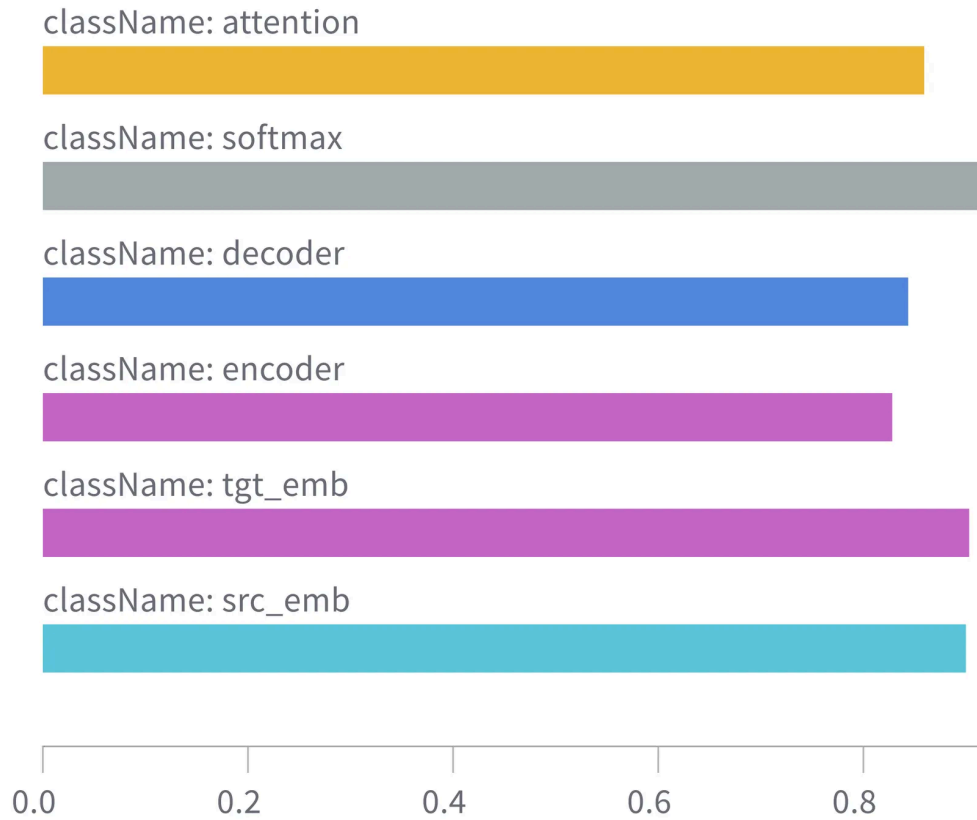
Comparison of random pruning coupled with retraining is included.



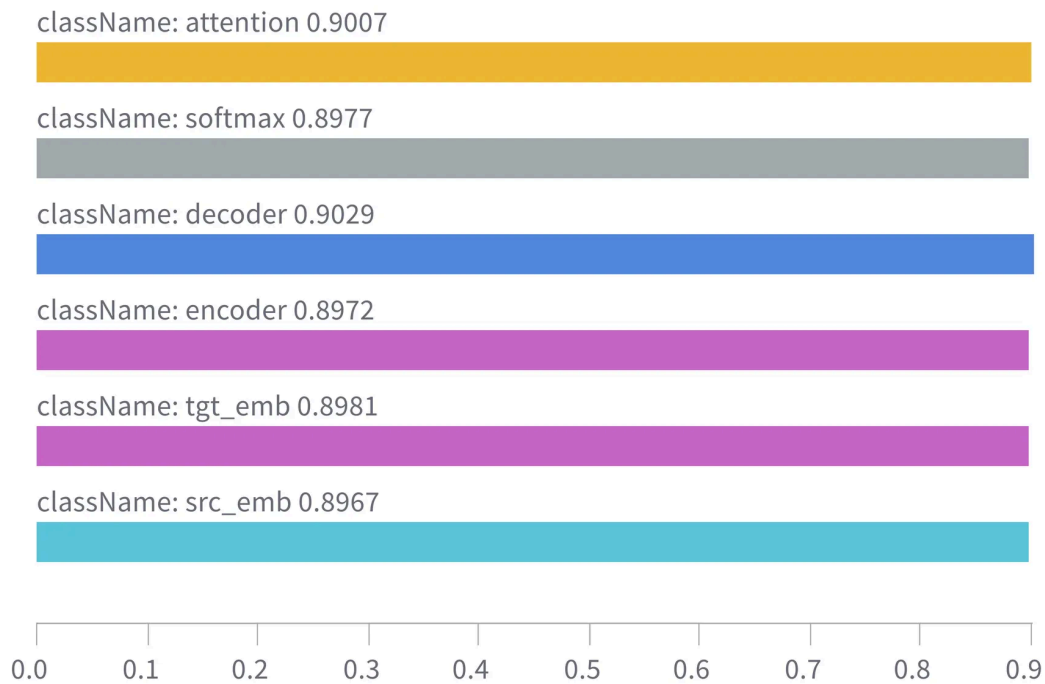
Graph shows that retraining is better, and it makes even random pruning much better. We can also use the sparse structure obtained during pruning to train sparse model from scratch. As we can see it is worse than retraining.

Percentage Pruned Per Class of Weights in Class-blind and Class-distribution for 90 percent pruning:

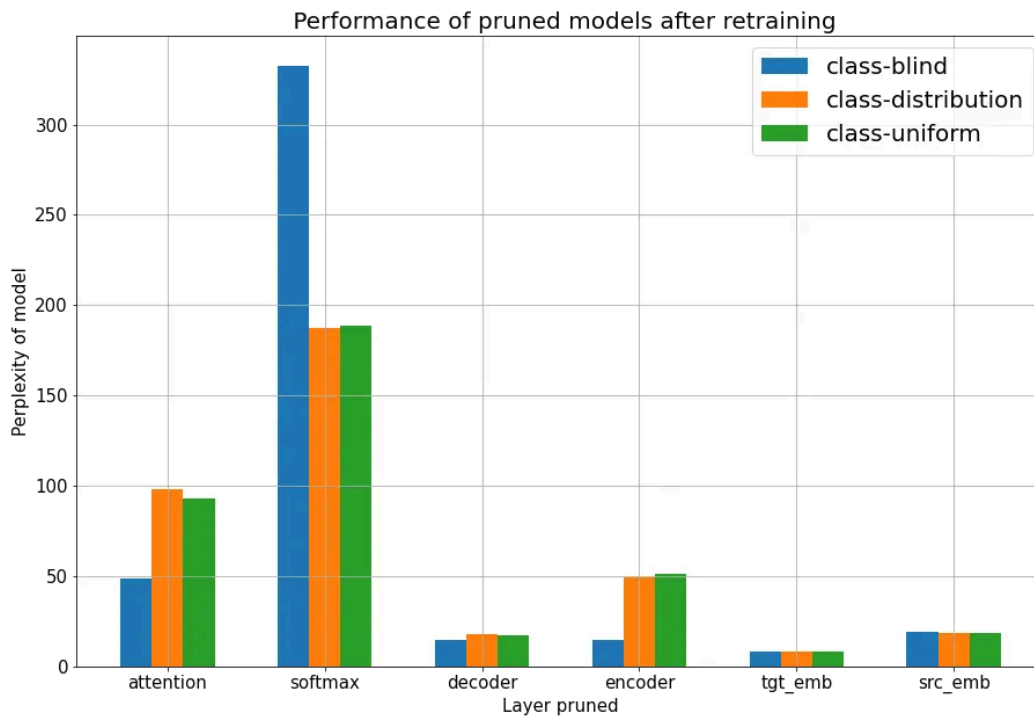
percentage_pruned



percentage pruned for class distribution

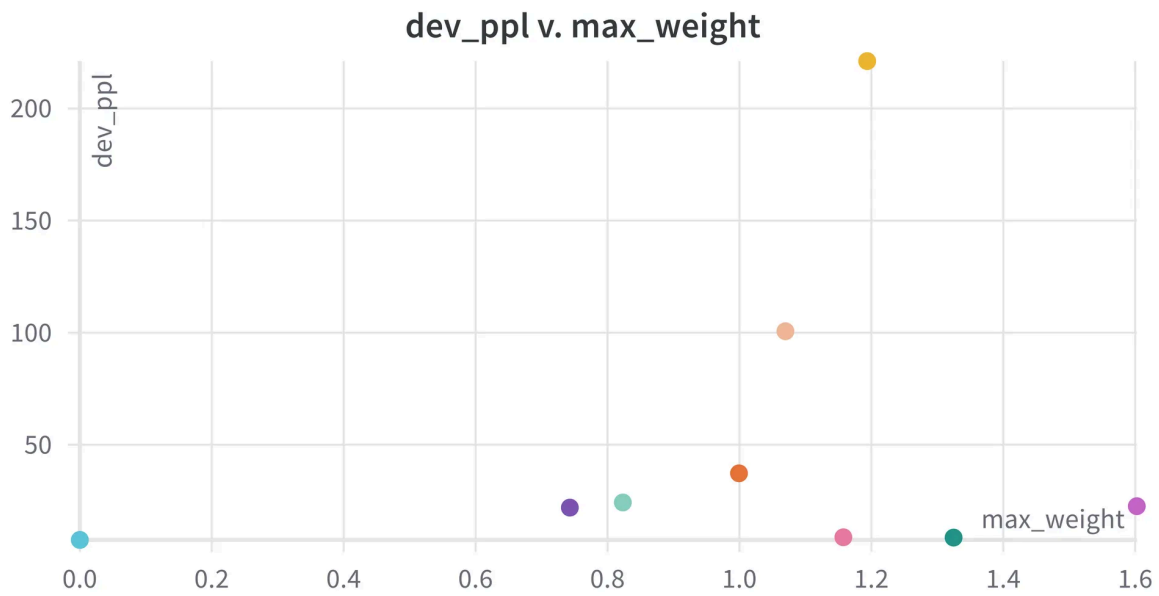


Perplexity Change with pruning of each class of weights:



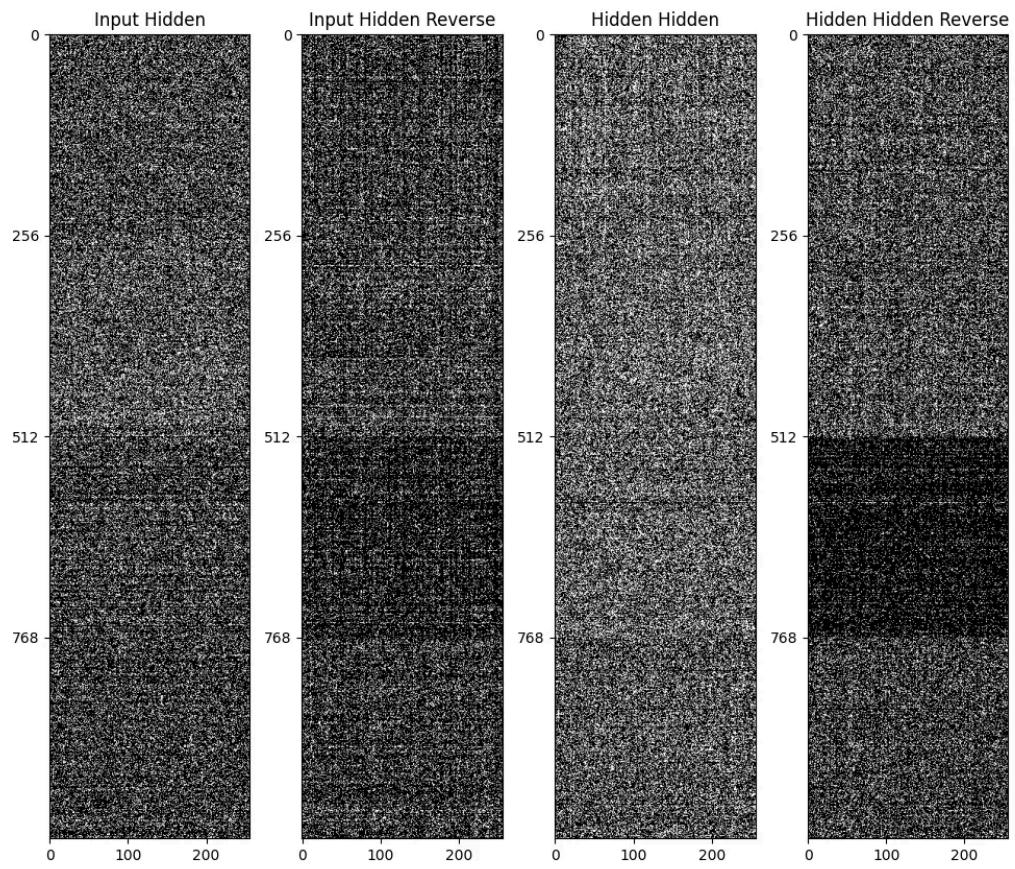
The graph shows that that some layers give higher perplexity change even though same number is pruned. This means those layers are more important and pruning them equally is not a good strategy. Interestingly last layers (softmax and attention) are more important than initial (target and source embeddings).

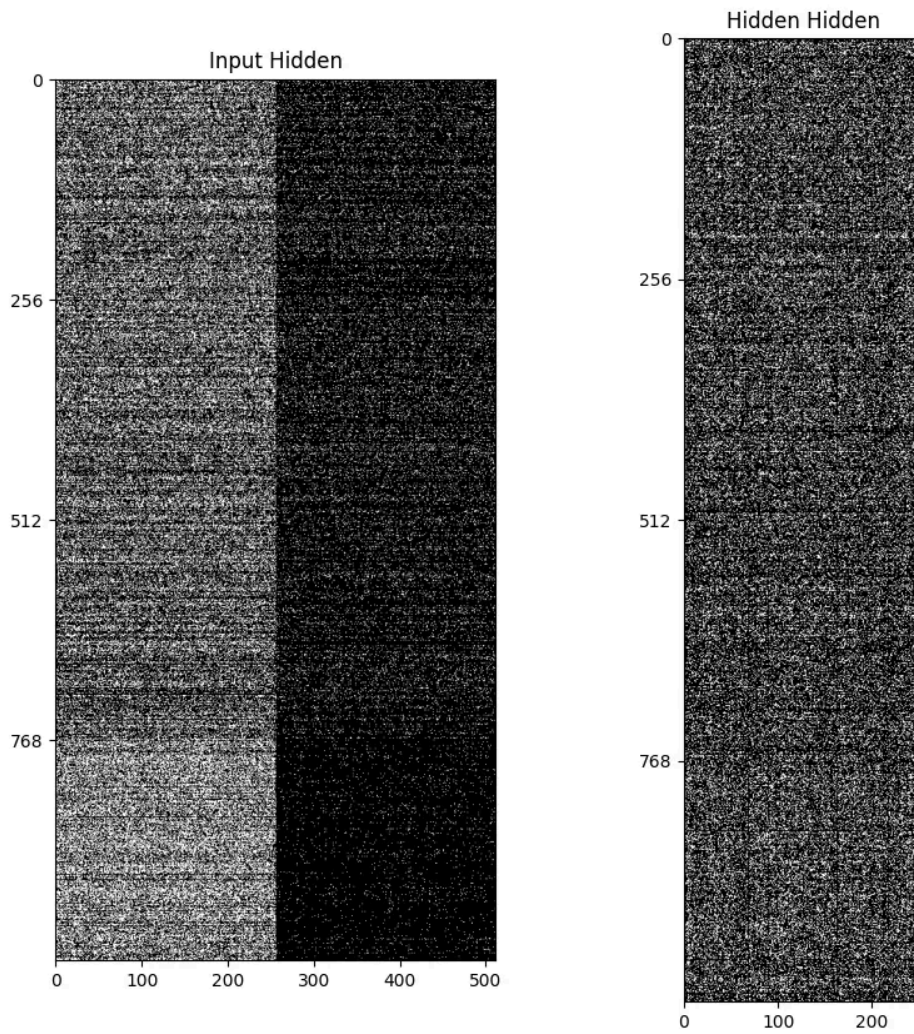
Change in perplexity vs max weight removed of that class



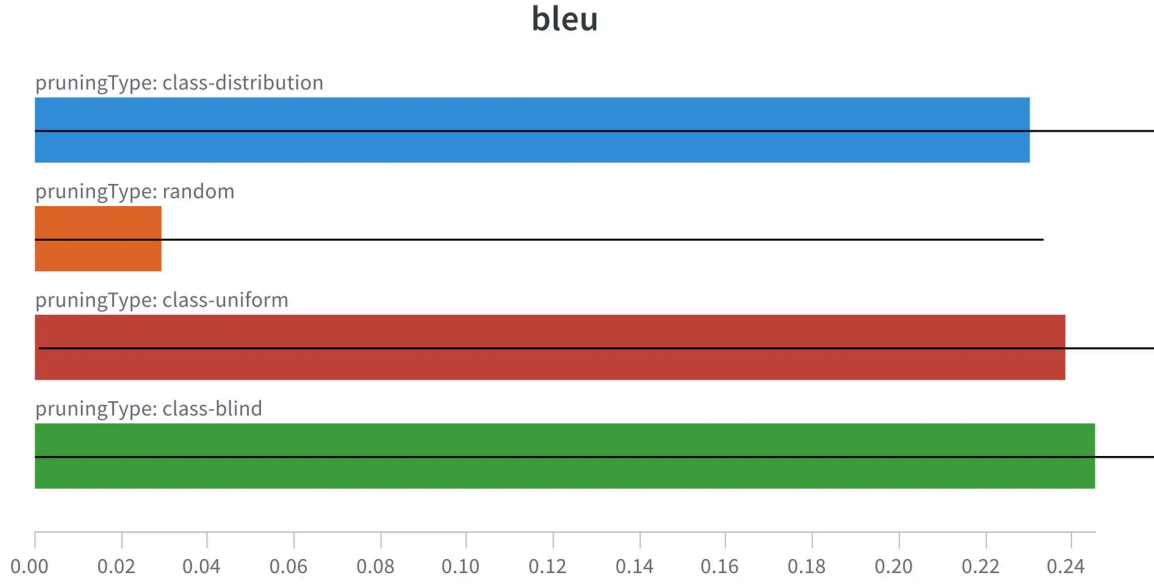
Encoder and Decoder Weights Visualisation

a) Encoder:





Median Bleu Score Comparison:



Comparision with other techniques (For LSTM)

Optimal Brain Damage

It a gradient pruning technique by LeCun et. al published in 1989 which started the field of pruning neural network weights.

Using Taylor's expansion we can approximate change in loss, dL as:

$$dL = \sum_i \frac{\partial L}{\partial w_i} dw_i + \frac{1}{2} \sum_i \left(\frac{\partial^2 L}{\partial w_i^2} \right) (dw_i)^2 + \frac{1}{2} \sum_{i \neq j} \frac{\partial^2 L}{\partial w_i \partial w_j} dw_i dw_j + O(n^3)$$

Now since the model has converged, the first term is zero and we ignore the interaction and higher order terms what we finally get is:

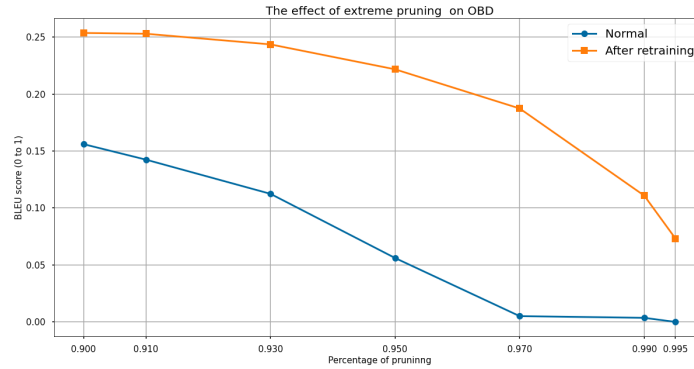
$$dL = \sum_i \left(\frac{\partial^2 L}{\partial w_i^2} \right) (dw_i)^2 \quad (2)$$

Second derivative of the loss functions with respect to parameters is used a saliency measure.

Why does this work?

Gradient pruning prunes the weights gradients with respect to whose are negligible as these weights do not cause more change the loss function so they must not be that important.

Effect of extreme pruning percentages on OBD:



Single Shot Network Pruning (SNIP) based on connection sensitivity:

Why is even there a need for pruning before training?

- It identifies saliency criterion based on connection sensitivity that identifies structurally important connections in the network for the given task before training starts.
- This eliminates the need for both retraining and the complex pruning schedule while making it *robust to architecture variations*.

$$\min_{\mathbf{c}, \mathbf{w}} L(\mathbf{c} \odot \mathbf{w}; \mathcal{D}) = \min_{\mathbf{c}, \mathbf{w}} \frac{1}{n} \sum_{i=1}^n \ell(\mathbf{c} \odot \mathbf{w}; (\mathbf{x}_i, \mathbf{y}_i)) ,$$

$$\text{s.t. } \mathbf{w} \in \mathbb{R}^m ,$$

$$\mathbf{c} \in \{0, 1\}^m , \quad \|\mathbf{c}\|_0 \leq \kappa ,$$

Saliency criterion: Connection sensitivity involves removing the connection and observing the change in the network's output (loss function value) compared to the original network. So it does not matter if the loss has converged, we want only to see how much change does removing a weight produce to loss.

It can be modeled in following way:

Here, c is the mask which says if the weight must be present or not. c is element wise multiplied with corresponding weight.

Ideally, we would want to remove each w_j once (by setting $c_j = 0$), to check for effect on loss:

$$\Delta L_j(\mathbf{w}; \mathcal{D}) = L(\mathbf{1} \odot \mathbf{w}; \mathcal{D}) - L((\mathbf{1} - \mathbf{e}_j) \odot \mathbf{w}; \mathcal{D}) ,$$

However, we cannot do that in reasonable time, so we approximate the effect by calculating $\frac{\delta L}{\delta c}$:

$$\frac{\delta L}{\delta c}$$

$$g_j(\mathbf{w}; \mathcal{D}) = \left. \frac{\partial L(\mathbf{c} \odot \mathbf{w}; \mathcal{D})}{\partial c_j} \right|_{\mathbf{c}=\mathbf{1}}$$

Note that it is gradient with respect to mask, and not original weights differentiating it with previous methods.

It involves pruning the network once at initialisation and then training normally.

Effect of initialisation:

Different initialisation tried are:

- Uniform initialisations (as recommended by authors for given LSTM).

- Kaiming uniform
- Kaiming normal
- Xavier uniform
- Xavier normal

epochs	initialisation	lr	percen	prunin	bleu
25	kaming_normal	0.0025	90	snip	0.2001
25	xavier_uniform	0.0025	90	snip	0.0814
25	xavier_normal	0.0025	90	snip	0.08209
25	uniform	0.0025	90	snip	0.1518
25	kaming_uniform	0.0025	90	snip	0.1929

Initialisation matters a lot since the connection sensitivity is performed on untrained weights.. It should be variance scaled for the method to not depend on architecture variations.

Effect of batch size to find saliency -

In SNIP results might also change with how much data we show, before actually calculating the gradients to find saliency.

The graph shows that just $\frac{50^{th}}{1000}$ portion of dataset is enough to make a good predictions on pruning weights.

Problems with SNIP:

- Connection sensitivity is sub-optimal as a criterion because the gradient of each weight might change dramatically after pruning due to complicated interactions between weights.
- Since SNIP only considers the gradient for one weight in isolation, it could remove connections that are important to the flow of information through the network.

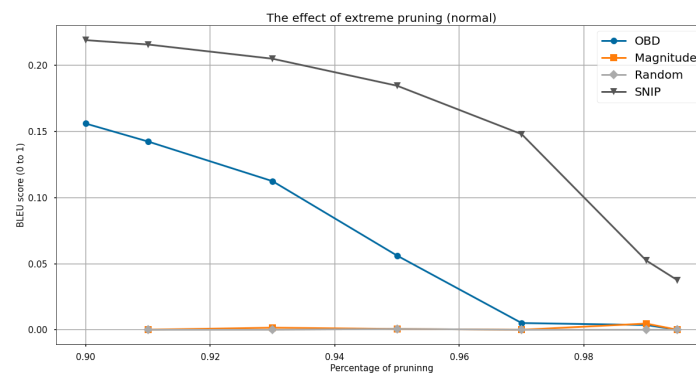
Different Experiment Performed For Comparison

Percentage Pruned vs class type pruned for 50 and 80 percent pruning

OBD and SNIP remove weights closer to start. This is actually good as later weights are considered to be more important.

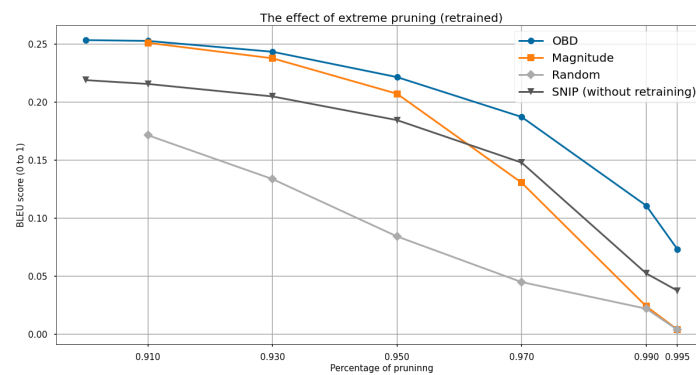
Effect of extreme pruning percentages comparison

a) Without retraining:



SNIP was pruned at initialisation, rest are pruned after training.

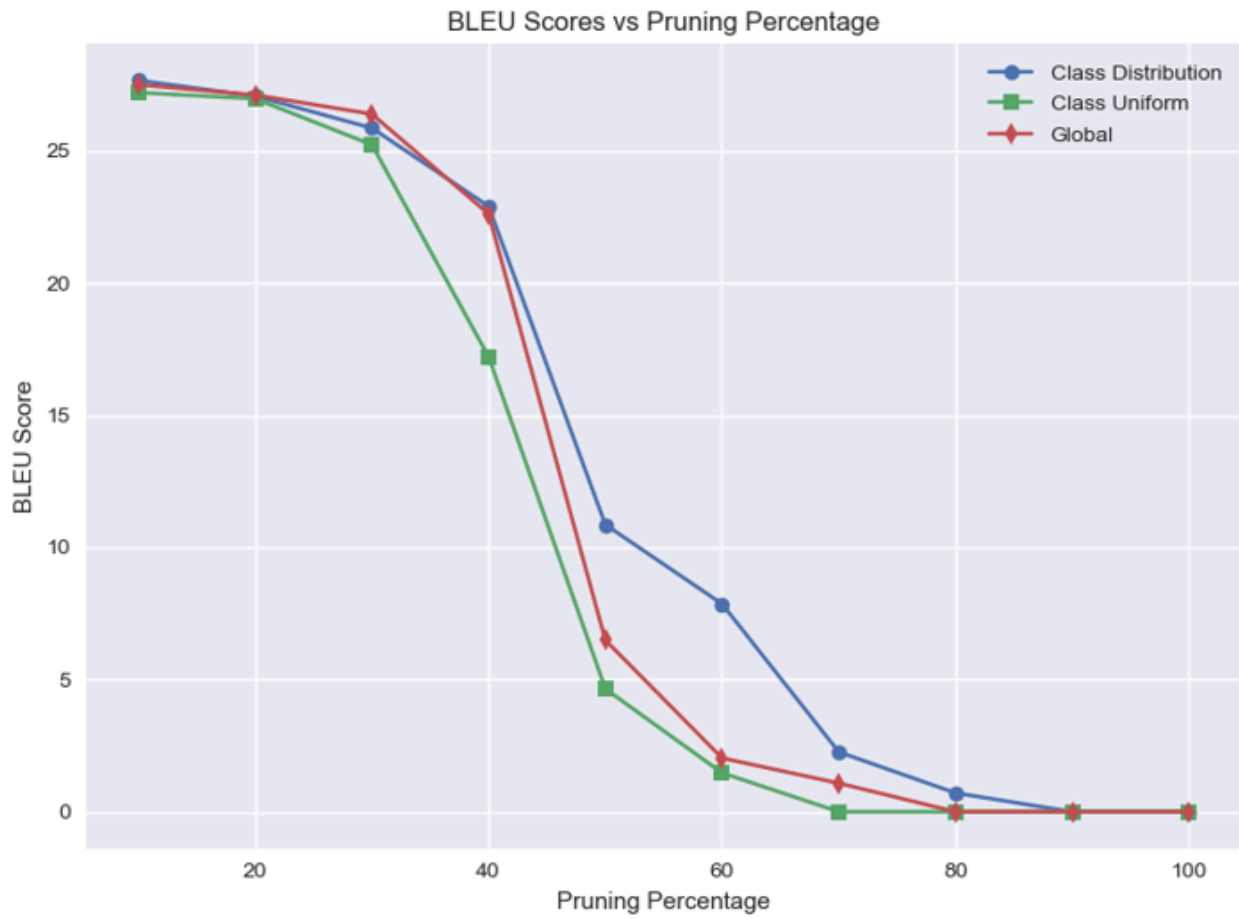
b) With retraining:

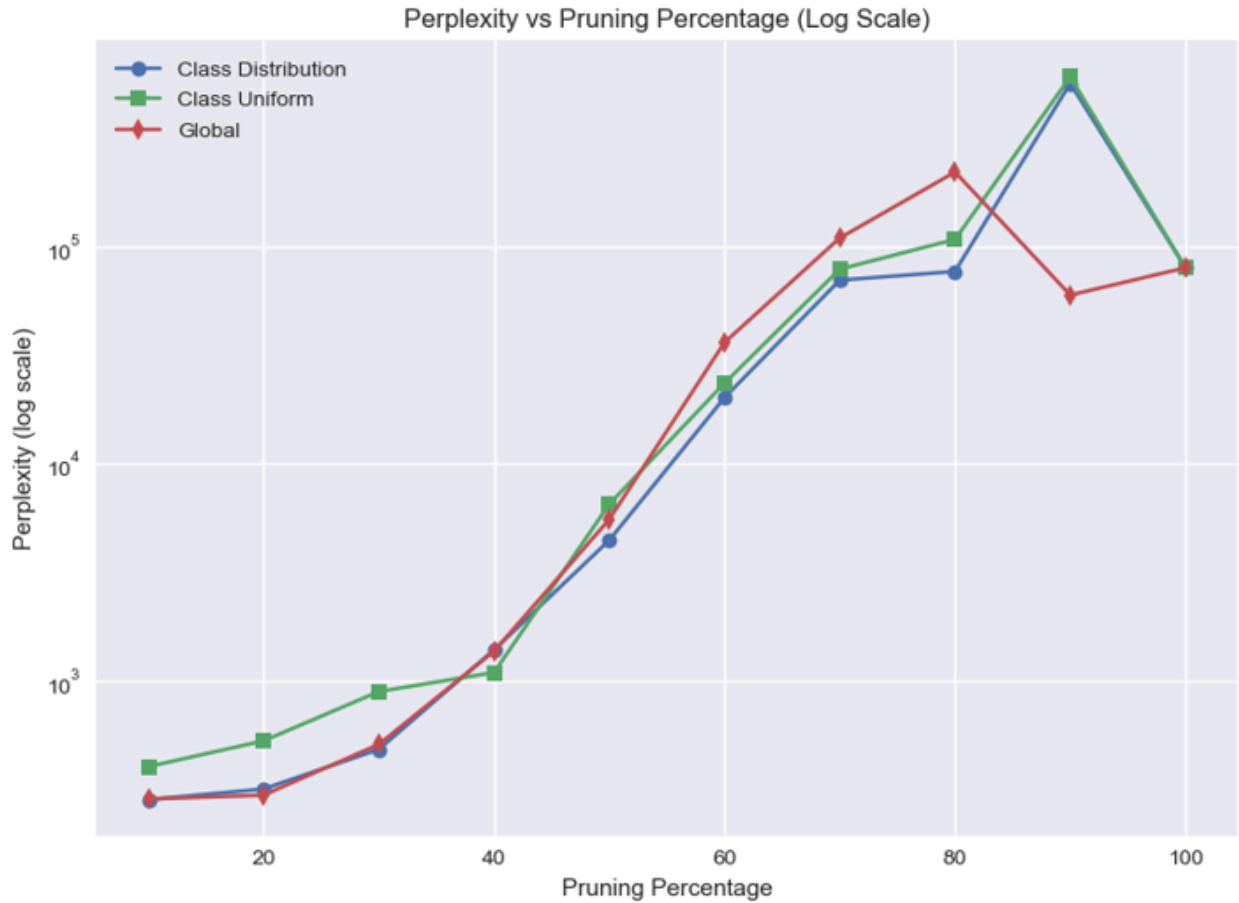


Top-3 pruning schemes are shown after retraining. SNIP is shown as usual.

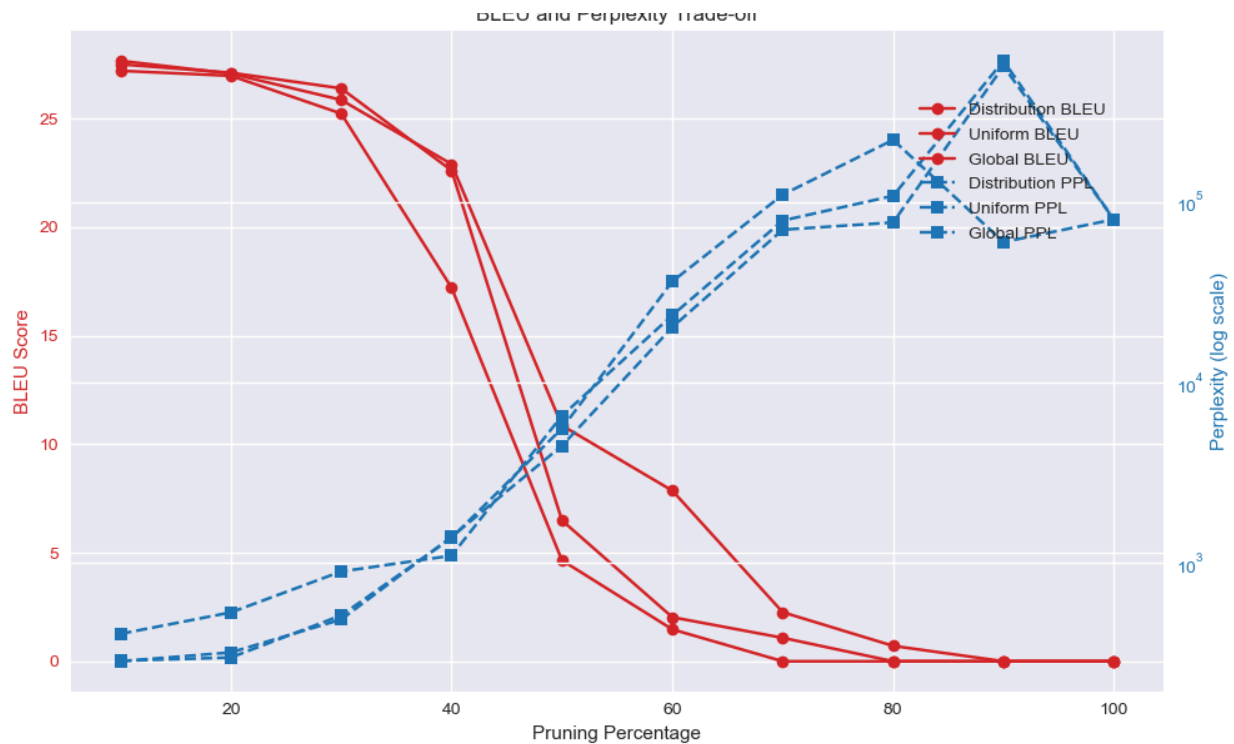
Both graphs seem to confirm that SNIP is good at extreme levels of pruning.

TRANSFORMER Results :





LSTMs process data sequentially, with less rigid weight specialization. Class-blind pruning treats all weights equally, avoiding over-pruning any single "important" part of the sequence. Transformers, however, rely on targeted attention, so blind pruning risks cutting crucial connections, while class-aware methods protect them. So , class distribution tends to perform the best here.

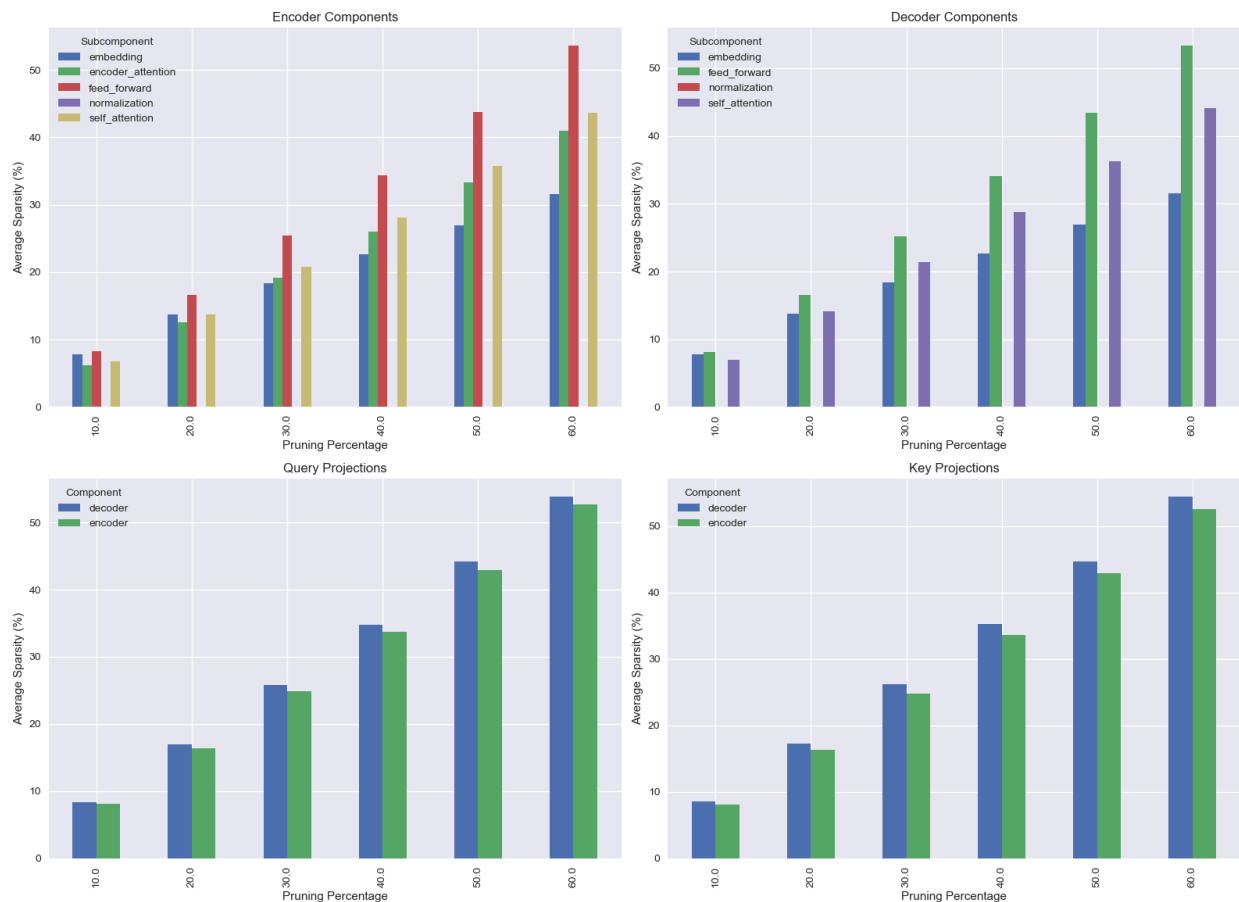


Best pruning level appears to be around **20–30%**, where BLEU scores remain high (~26–27) and perplexity is still low. This suggests that mild pruning reduces model redundancy without hurting performance, offering a good compression-performance tradeoff.

From **40% onward**, BLEU drops sharply, while perplexity increases exponentially—especially past 50%, indicating the model is losing critical parameters.

Distribution by sparsity :

Class Distribution

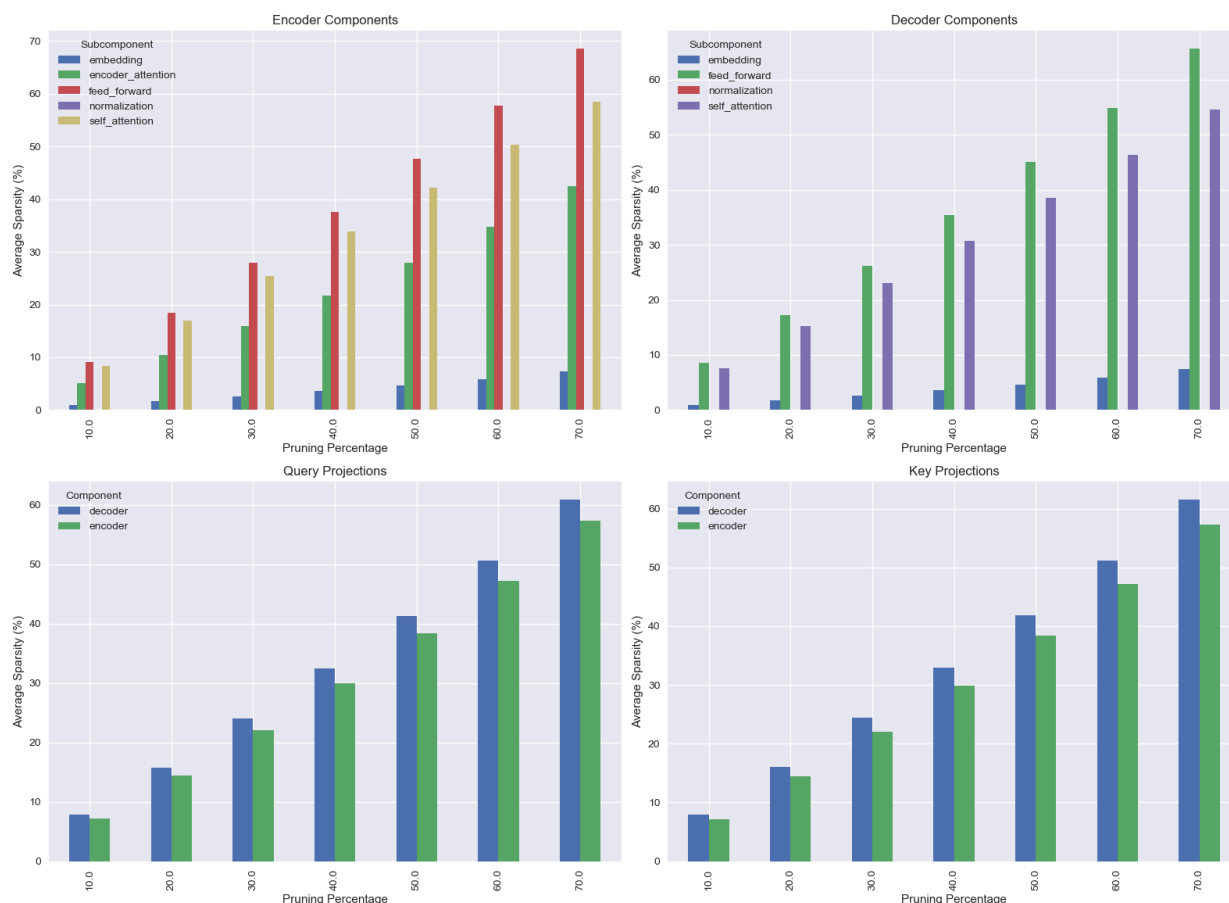


Encoder Components (top-left): The **feed-forward** and **self-attention** layers show the highest sparsity at higher pruning levels (up to ~50%), indicating they are major contributors to the prunable parameter count. Embeddings and normalization layers remain less sparse, likely due to their smaller size or sensitivity to pruning.

Decoder Components (top-right): Similar to the encoder, **feed-forward layers** dominate in sparsity, followed by self-attention. Interestingly, **normalization** layers stay consistently less sparse, suggesting their role in model stability discourages heavy pruning.

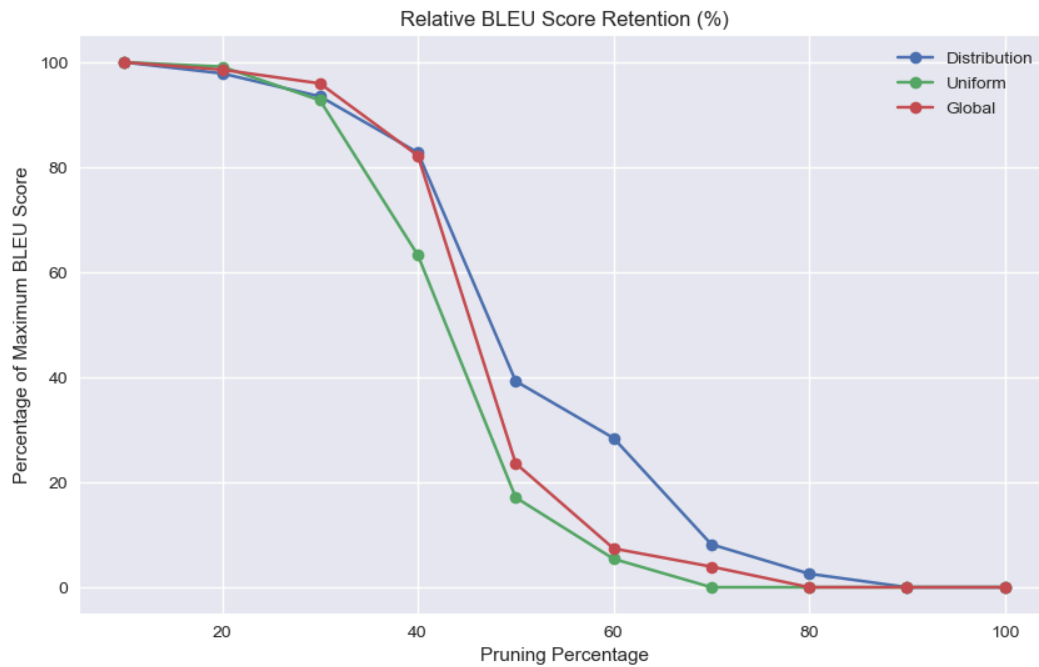
Query and Key Projections (bottom plots): Sparsity increases steadily in both encoder and decoder query/key projections, with both components showing similar trends. By 60% pruning, both query and key projections reach close to 50% sparsity, confirming these layers are heavily pruned—possibly due to redundancy in attention heads.

Global



- **Feed-forward layers** are still pruned the most, reaching over 60% sparsity at high pruning levels, similar to before.
- However, **normalization layers** and **embeddings** are now affected significantly more than in the previous setup. For example, embedding sparsity crosses 20% in some cases—unlike before, where it was largely preserved.
- This reflects the *structure-agnostic* nature of global pruning—it doesn't spare sensitive or smaller subcomponents, potentially hurting performance.

Bleu score Retention



Low pruning (10–30%): All three methods retain >95% of the original BLEU score, showing that early pruning has minimal impact on translation quality. Distribution-based pruning slightly outperforms the others at 30%.

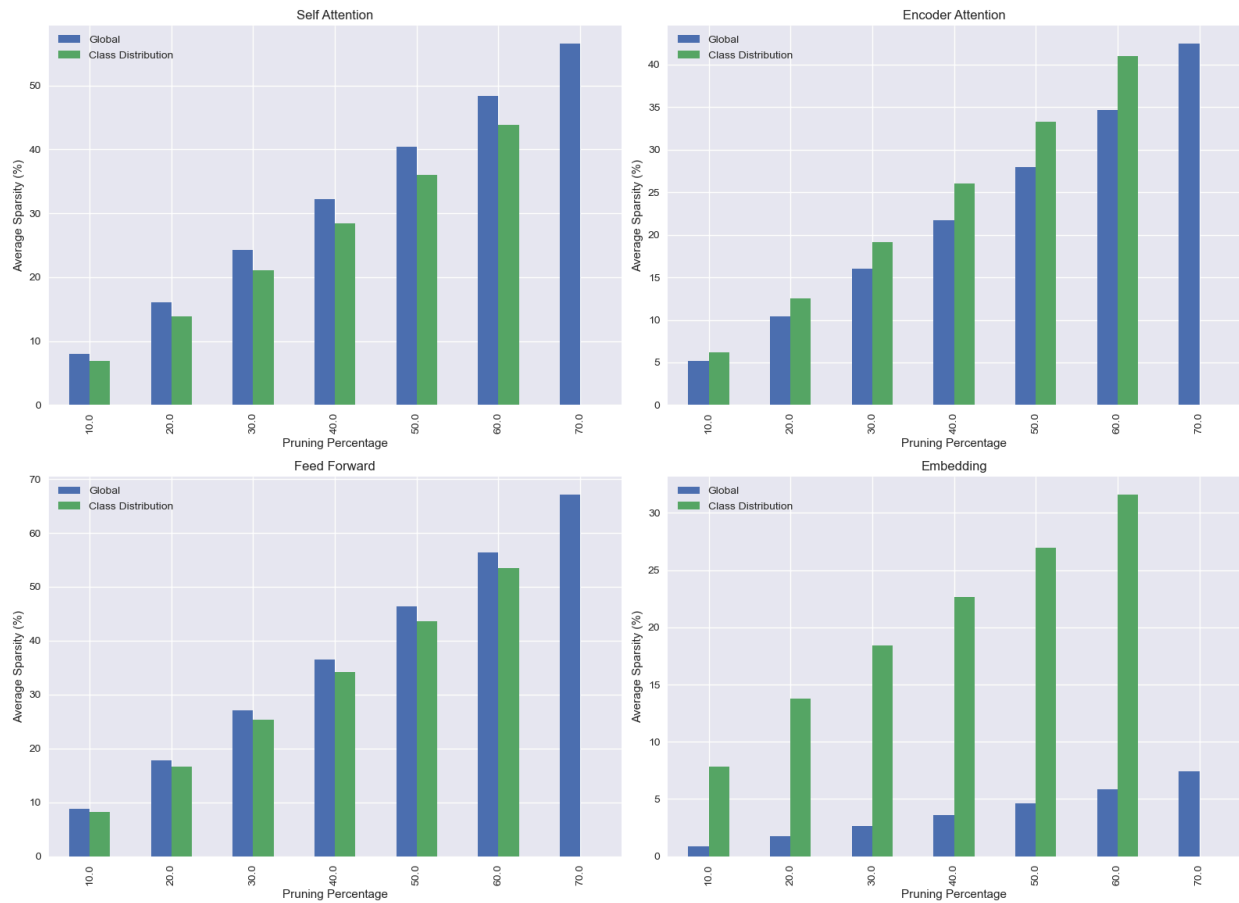
Mid pruning (40–60%):

- This is where divergence begins. **Distribution pruning** retains significantly more BLEU score (~40–30%) compared to **Global (~25%)** and especially **Uniform (~17%)**, which drops sharply.
- Suggests that **class-aware pruning** (distribution-based) is better at preserving functional capacity under medium sparsity.

High pruning (>70%):

- All methods converge to near-zero BLEU retention, indicating that aggressive pruning removes essential parameters regardless of strategy.
- **Uniform pruning** consistently underperforms throughout, likely due to blindly pruning equal amounts across layers without considering importance.

Component wise comparison



1. Self-Attention

- Both strategies show a roughly linear increase in sparsity with pruning percentage.
- Global pruning consistently results in slightly higher sparsity than class-distribution pruning.
- Suggests that self-attention is aggressively pruned under global strategy, possibly overlooking class-wise significance.

2. Encoder Attention

- Class-distribution pruning yields higher sparsity than global at moderate-to-high pruning levels (40–70%).

- Indicates this method concentrates sparsity more heavily in this component, possibly identifying it as less critical compared to others.

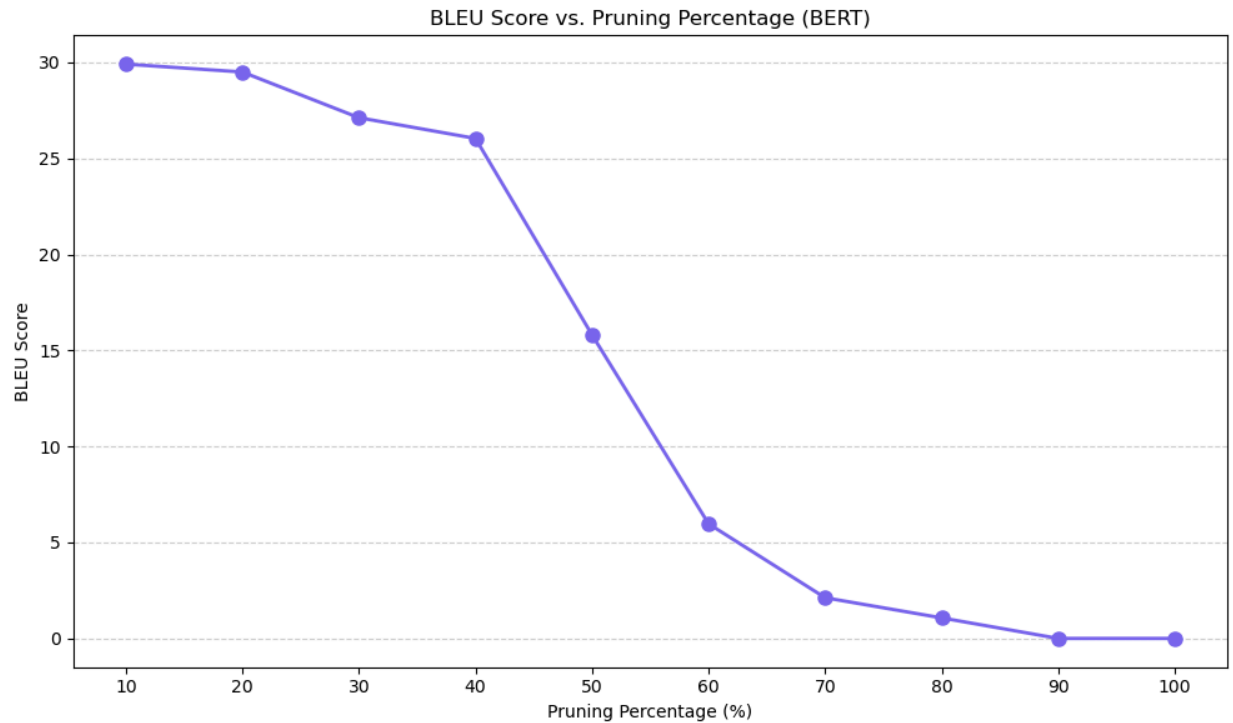
3. Feed Forward

- Sparsity under class-distribution exceeds that of global pruning especially at 40% and 60% levels.
- Shows that class-aware pruning targets feedforward layers more, aligning with known overparameterization in FFN sublayers of Transformers.

4. Embedding

- Striking divergence: class-distribution pruning yields significantly higher sparsity (~50%+ at 60–70%), while global pruning barely touches the embedding layer.
- Suggests global pruning over-protects embeddings, while class-aware strategy identifies and prunes redundant embedding dimensions more effectively

BERT Results



We noticed that using Bert embeddings for the transformer did not result in a major increase in performance.

Link to code

https://github.com/Vanshika-Dhingra/inlp_project